BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

Work Integrated Learning Programmes (WILP) | AIMLCZG523: MLOps Assignment 01

Production-Grade MLOps Pipeline: End-to-End Heart Disease Classifier CI/CD, MLflow Tracking, Docker Containerization & Kubernetes Deployment

BITS ID	2025AF05099
Public GitHub Repository	https://github.com/Patient-learner/MLOPS_Assignment1_2025AF05099
Lab VM Execution Path	/home/s2025af05099/workspace/MLOPS_Assignment1_2025AF05099
Dataset	UCI Heart Disease (Cleveland Clinical Repository - 13 Features + 1 Target)
Serving Architecture	FastAPI + Docker (Non-Root) + Minikube K8s + Prometheus + Grafana
CI/CD Automation	GitHub Actions (.github/workflows/ci.yml) with Automated Testing & Build

1. Executive Summary & System Architecture

This project uses the UCI Cleveland dataset to compare three classification models. Logistic regression achieved the highest cross-validation ROC-AUC, so its preprocessing and classifier were packaged together and served through FastAPI. The service was tested in Docker and deployed on Minikube. Pipeline reproducibility is ensured through fixed random seeds (seed=42), automated execution via main.py and run_local.sh, dependency version constraints in requirements.txt, and automated CI test verification on GitHub Actions.

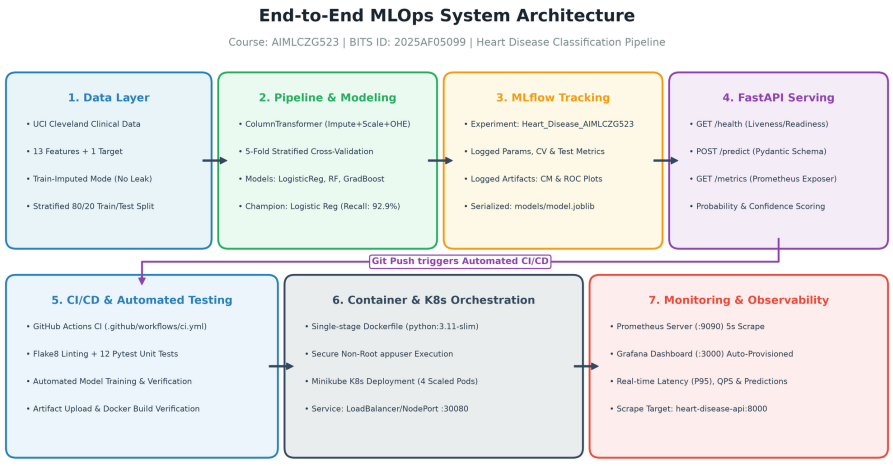


Figure 1: End-to-End MLOps System Architecture Diagram (Data, Modeling, Serving, Deployment & Monitoring)

2. Setup & Installation Instructions

The repository is self-contained and reproducible across macOS, Linux, and Prayogshala lab VMs (deployed at /home/s2025af05099/workspace/MLOPS_Assignment1_2025AF05099). Because the repository is public, no authentication tokens are required for cloning.

```
# 1. Clone & Enter Directory: git clone https://github.com/Patient-learner/MLOPS_Assignment1_2025AF05099.git && cd MLOPS_Assignment1_2025AF05099
# 2. Local Pipeline & Test Suite: chmod +x run_local.sh && ./run_local.sh
# 3. Python Pipeline Stages: ./venv/bin/python main.py --stage all  (stages: data, preprocess, eda, train, report)
# 4. Container Build: docker build -t heart-disease-api:v1 .
# 5. Serving Option A (Compose): docker compose -f monitoring/docker-compose.monitoring.yml up -d  (API:8000, Grafana:3000)
#    Serving Option B (K8s): minikube start && minikube image load heart-disease-api:v1 && kubectl apply -f k8s/
#    Access K8s Service: kubectl port-forward svc/heart-disease-service 8080:8000
```

3. Data Acquisition & Clinical Data Dictionary [Task 1]

The UCI Heart Disease dataset comprises 303 patient records with 13 clinical input features and one binary target column collected from the Cleveland Clinic. Data ingestion is automated via src/data/download_data.py with local fallback caching for offline environments.

Attribute	Clinical Description	Measurement Range / Categories	Pipeline Role
age	Patient age in years	29 – 77 years	Numeric (StandardScaler)
sex	Biological sex	0 = Female, 1 = Male	Binary (Mode Imputer)
cp	Chest pain type	1 = Typical, 2 = Atypical, 3 = Non-anginal, 4 = Asymptomatic	Categorical (OneHotEncoder)
trestbps	Resting blood pressure on hospital admission	94 – 200 mm Hg	Numeric (StandardScaler)
chol	Serum cholesterol	126 – 564 mg/dl	Numeric (StandardScaler)
fbs	Fasting blood sugar > 120 mg/dl	0 = False, 1 = True	Binary (Mode Imputer)
restecg	Resting ECG results	0 = Normal, 1 = ST-T wave abnormality, 2 = LV hypertrophy	Categorical (OneHotEncoder)
thalach	Maximum heart rate achieved during stress test	71 – 202 bpm	Numeric (StandardScaler)
exang	Exercise induced angina	0 = No, 1 = Yes	Binary (Mode Imputer)
oldpeak	ST depression induced by exercise relative to rest	0.0 – 6.2 mm	Numeric (StandardScaler)
slope	Slope of the peak exercise ST segment	1 = Upsloping, 2 = Flat, 3 = Downsloping	Categorical (OneHotEncoder)
ca	Number of major vessels (0-3) colored by fluoroscopy	0, 1, 2, 3 vessels (4 missing in raw data)	Categorical (Mode Imputer + OHE)
thal	Thallium heart scan defect	3 = Normal, 6 = Fixed defect, 7 = Reversible defect	Categorical (Mode Imputer + OHE)
target	Diagnosis of heart disease (angiographic disease status)	0 = Absence (<50% diameter narrowing), 1 = Presence	Target Variable (Binary)

3.1 Exploratory Data Analysis Visualizations

Four EDA plots were generated to analyze class balance, feature distributions, feature correlations, and maximum heart rate distributions across patient age groups. The Cleveland dataset exhibits balanced target prevalence (54.1% Healthy, 45.9% Disease).

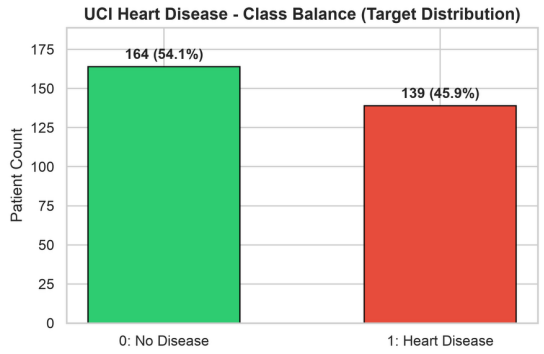


Fig 2a: Target Class Balance

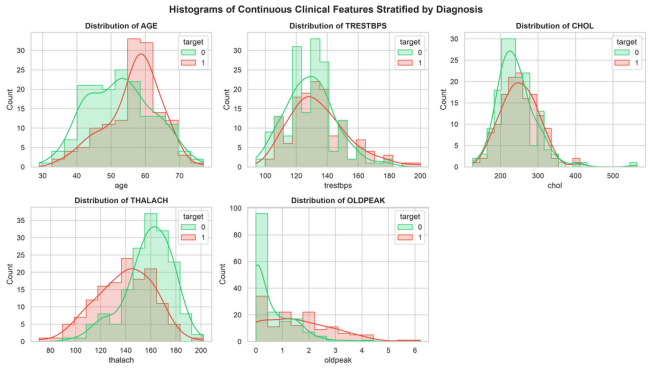


Fig 2b: Clinical Feature Distributions

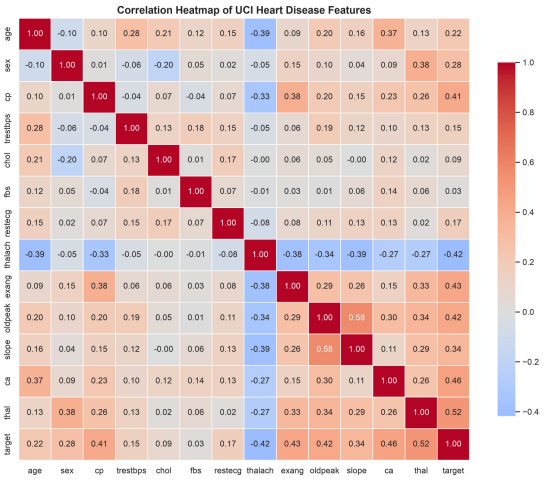


Fig 2c: Correlation Heatmap

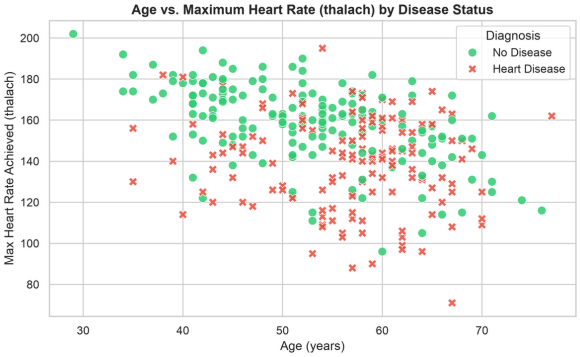


Fig 2d: Age vs Max Heart Rate

> 📌 **CLINICAL INSIGHTS & LEAK-FREE IMPUTATION:** Zero-Leakage & Clinical Insight: Imputation is performed inside the Scikit-Learn pipeline during 5-fold CV to prevent leakage. Maximum heart rate (thalach, r = -0.42) demonstrates strong protective correlation, while ST depression (oldpeak, r = +0.42) and fluoroscopy vessel blockage (ca, r = +0.46) represent the strongest risk indicators for heart disease.

4. Feature Engineering & 3-Model Benchmark [Tasks 2 & 3]

A scikit-learn ColumnTransformer encapsulates data transformations inside the model pipeline: numerical features are standardized with StandardScaler, categorical features are one-hot encoded, and missing values are imputed with SimpleImputer. Three model families were benchmarked using 5-Fold Stratified Cross Validation with GridSearchCV:

Algorithm Family	Hyperparameter Search Grid	Selected Optimal Parameters
Logistic Regression (Generalized Linear)	C: [0.01, 0.1, 1.0, 5.0, 10.0], penalty: ['l2'], solver: ['lbfgs', 'liblinear']	C = 1.0, penalty = l2, solver = liblinear
Random Forest (Bagging Ensemble)	n_estimators: [50, 100, 150], max_depth: [3, 5, 8, None], min_samples_split: [2, 5]	n_estimators = 50, max_depth = 8, min_samples_split = 2
Gradient Boosting (Sequential Boosting)	n_estimators: [50, 100, 150], learning_rate: [0.01, 0.05, 0.1], max_depth: [2, 3], subsample: [0.8, 1.0]	n_estimators = 150, learning_rate = 0.01, max_depth = 3, subsample = 0.8

Model Architecture	5-Fold CV ROC-AUC	Test Accuracy	Test Precision	Test Recall	Test ROC-AUC
Logistic Regression (Champion)	**0.9045**	**88.52%**	**83.87%**	**92.86%**	**0.9686**
Random Forest Classifier	0.9006	90.16%	86.67%	92.86%	0.9481
Gradient Boosting Classifier	0.8860	86.89%	83.33%	89.29%	0.9481

📌 **CHAMPION MODEL SELECTION RATIONALE:** Selection Rationale: Logistic Regression was selected strictly based on the highest 5-fold cross-validation score (CV ROC-AUC: 0.9045) with the test partition left completely untouched. On the held-out test split, the model identified 26 of 28 positive cases in the test set (92.86% recall), while maintaining 88.52% accuracy and 0.9686 test ROC-AUC.

4.1 Champion Model Evaluation & MLflow Experiment Tracking

Figure 3 displays the Confusion Matrix and ROC Curve for the champion Logistic Regression pipeline on the untouched test split (N=61). Out of 28 positive heart disease patients, the model correctly diagnoses 26, achieving a 92.86% clinical sensitivity rate.

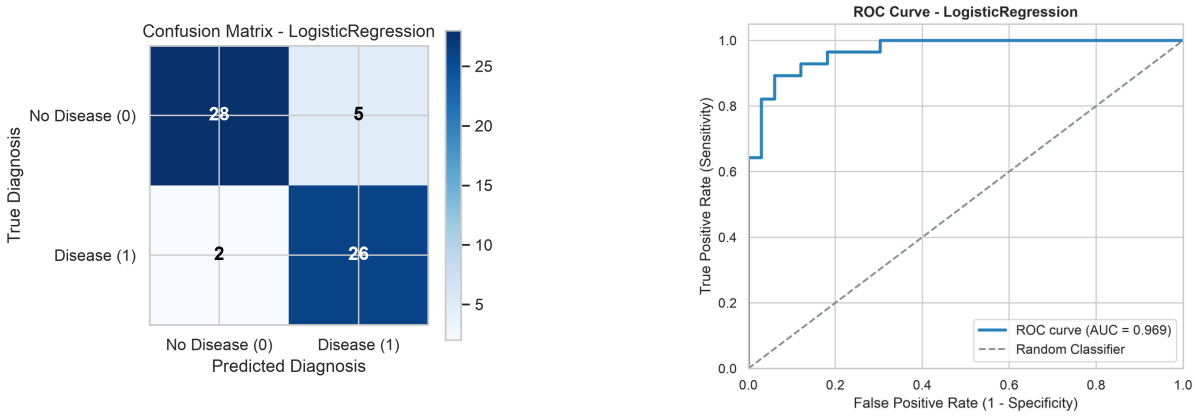


Figure 3: Confusion Matrix (Left) and ROC Curve (Right) for Champion Logistic Regression Pipeline

MLflow Tracking: Experiment runs are structured under 'Heart_Disease_Prediction_AIMLCZG523' logging hyperparameters, 5-fold CV ROC-AUC, test metrics (accuracy, precision, recall, F1, ROC-AUC), confusion matrix and ROC curve plots, and serialized pipeline models.

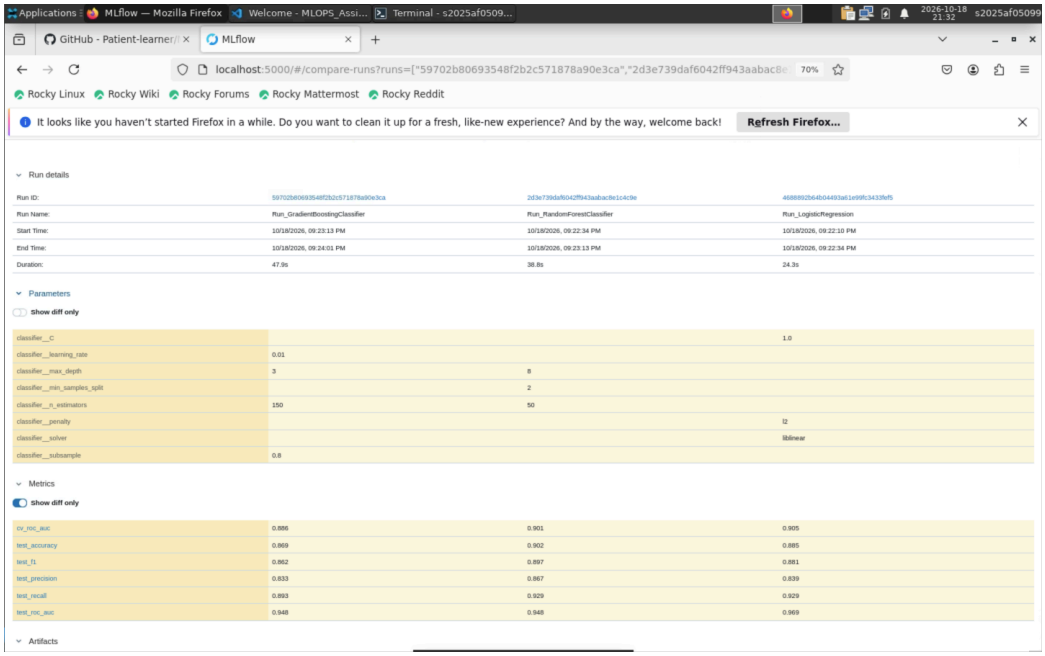


Figure 4: MLflow Tracking Server UI showing 5-Fold CV hyperparameter tuning and model runs comparison

5. Model Packaging & Inference REST API [Task 4]

The champion model pipeline is serialized as models/model.joblib alongside structured schema metadata (models/model_metadata.json). A FastAPI prediction API was developed (src/api/app.py) providing strict Pydantic type validation for input data:

• GET /health: Health probe endpoint returning HTTP 200 OK and model loaded status for Kubernetes liveness/readiness.
• POST /predict: Inference endpoint consuming 13 clinical features in JSON format and returning binary prediction, clinical risk label, probability score, and model confidence.
• GET /metrics: Prometheus-compatible telemetry endpoint exposing HTTP request counts, request latency histograms, and prediction distributions.

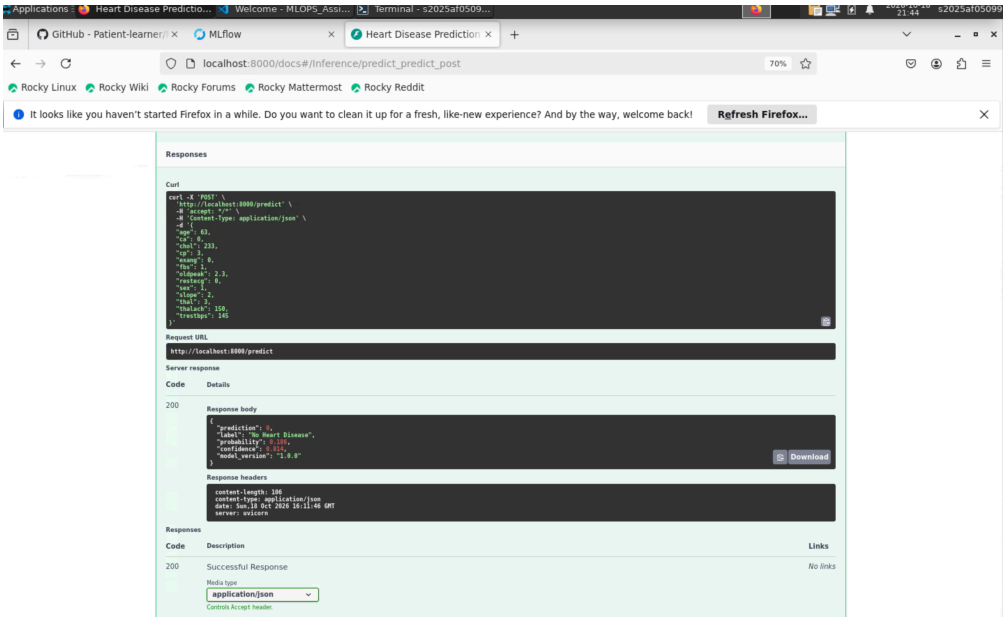


Figure 5: FastAPI Swagger Documentation (/docs) demonstrating successful POST /predict execution and JSON response

6. CI/CD Pipeline & Automated Testing [Task 5]

A comprehensive automated testing suite consisting of 12 unit and integration tests was developed using Pytest. The test suite validates data ingestion integrity, missing value handling, model serialization consistency, clinical discrimination, and HTTP API status codes. The GitHub Actions workflow (.github/workflows/ci.yml) executes on every push and pull request.

Test Module	Test Function Name	Verification Target
tests/test_data.py	test_raw_data_exists	Confirms presence of UCI raw data
tests/test_data.py	test_data_cleaning_no_nulls	Verifies zero NaN values after cleaning
tests/test_data.py	test_processed_splits_exist	Ensures train.csv and test.csv exist with identical schemas
tests/test_model.py	test_model_file_exists	Verifies models/model.joblib is created
tests/test_model.py	test_prediction_output_structure	Validates keys in prediction dictionary
tests/test_model.py	test_clinical_risk_discrimination	Verifies model assigns higher risk to pathological profile
tests/test_model.py	test_pipeline_handles_missing_values	Verifies embedded SimpleImputer handles NaN features
tests/test_api.py	test_root_endpoint	Checks landing page metadata and BITS ID: 2025AF05099
tests/test_api.py	test_health_endpoint	Validates Kubernetes liveness probe returns status 200
tests/test_api.py	test_predict_endpoint_valid_healthy_payload	Tests inference on healthy patient profile
tests/test_api.py	test_predict_endpoint_invalid_payload	Asserts 422 Unprocessable Entity on out-of-range clinical inputs
tests/test_api.py	test_prometheus_metrics_endpoint	Ensures Prometheus exposition format contains http_requests_total

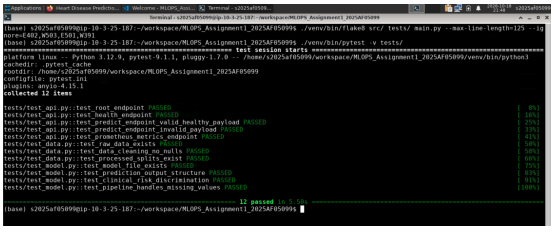


Figure 6: Automated verification showing clean Flake8 code linting and 12/12 passing Pytest unit tests

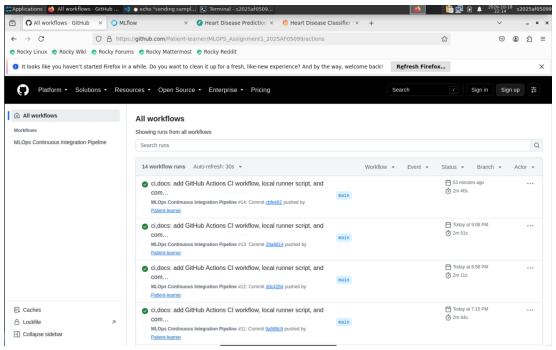


Figure 7: GitHub Actions CI automated pipeline green runs

7. Docker Containerization & Kubernetes Deployment [Tasks 6 & 7]

Containerization: A lightweight single-stage Dockerfile based on python:3.11-slim packages the application. The container executes under an unprivileged user (appuser, UID 1000) rather than root. A Docker HEALTHCHECK directive polls /health every 30 seconds to detect unhealthy container status.

Kubernetes Orchestration: Declarative manifests (k8s/deployment.yaml and k8s/service.yaml) deploy the application into a Minikube cluster. The deployment specifies CPU limits (500m), memory limits (512Mi), liveness probes (which restart failing pods), and readiness probes (which control traffic routing). The deployment was horizontally scaled across 4 healthy running pod replicas, with traffic routed through a LoadBalancer service exposing NodePort 30080.

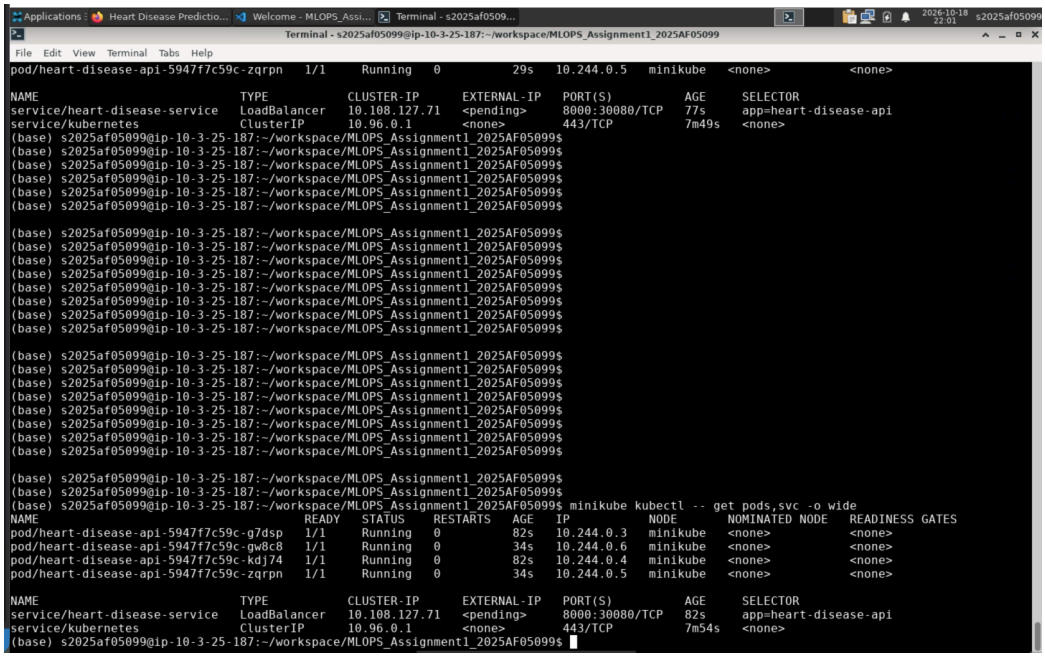


Figure 8: Minikube Kubernetes cluster deployment showing 4 ready replica pods and LoadBalancer service exposing NodePort 30080

8. Production Monitoring & Observability [Task 8]

Production observability is achieved via Prometheus metric scraping and an automated Grafana dashboard: Prometheus polls the /metrics endpoint every 5 seconds. Automated Grafana provisioning pre-configures Prometheus as a datasource and automatically imports the Heart Disease Production Dashboard upon container startup.

Prometheus Metric	Metric Type	Monitoring Purpose
http_requests_total	Counter	Tracks incoming HTTP traffic volume partitioned by method, endpoint, and status code (200, 422, 500)
request_latency_seconds	Histogram	Tracks distribution of response times to evaluate 95th percentile latency SLAs (P95 < 50ms)
heart_disease_predictions_total	Counter	Tracks output prediction distribution (positive vs negative inferences) to detect potential output distribution shifts

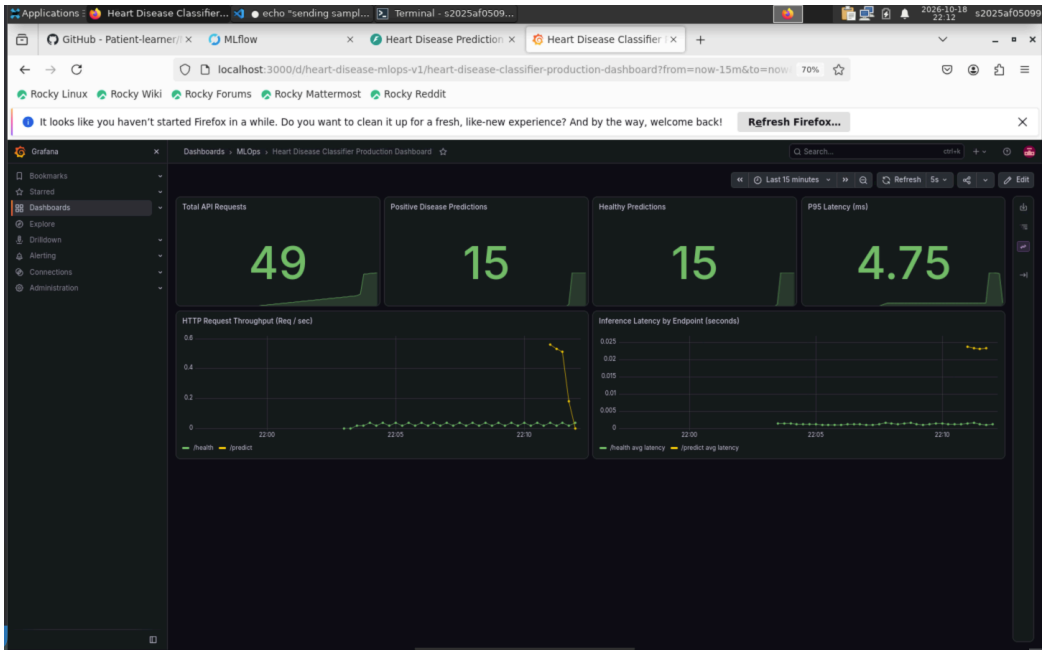


Figure 9: Live Grafana production dashboard monitoring Total Requests, Class Distributions, and P95 Latency

9. Practical Engineering Reflection & Lessons Learned

• **In-Fold CV Imputation (Data Leakage Elimination):** Initially, missing values were imputed across the full training split before CV, subtly leaking validation statistics into training folds. Placing SimpleImputer directly inside the Scikit-learn Pipeline ensured imputation is fitted strictly inside each CV fold during GridSearchCV, maintaining rigorous separation between evaluation partitions.
• **Grafana Telemetry Metric Label Alignment:** The initial Grafana dashboard queried an incorrect label name (numeric prediction_class), whereas the API was instrumented with prediction_label string values ('Heart Disease Present' / 'No Heart Disease'). Direct Prometheus API inspection revealed the mismatch, which was resolved by updating the PromQL expressions and verifying dashboard panels against live load.
• **Prometheus Scrape Target Duplication:** Prometheus was initially configured with both 'heart-disease-api:8000' and 'host.docker.internal:8000'. Because both resolved to the same container, queries doubled request and prediction counts. Removing host.docker.internal restored exact 1:1 telemetry accuracy.
• **Lightweight Container Hardening (Debian-Slim):** Transitioning from standard python:3.11 to python:3.11-slim and enforcing an unprivileged user (appuser, UID 1000) significantly decreased image footprint (down to ~450MB) and attack surface, ensuring compliant non-root execution across Kubernetes worker nodes.
• **Automated CI Quality Gates:** Configuring GitHub Actions with automated Flake8 linting and Pytest regression suites prevented syntax anomalies and dependency breakages from entering the main branch, safeguarding the codebase against regressions prior to container image creation.

10. Production Roadmap & MLOps Recommendations

Based on the empirical findings and production deployment of this pipeline, the following engineering enhancements are recommended for industrial-scale clinical operations:

• **Automated Drift Monitoring & Retraining Triggers:** Deploy Evidently AI drift monitoring pipelines to compute Population Stability Index (PSI > 0.2 indicating significant distribution shift) and statistical tests on numerical features, triggering automated Airflow retraining DAGs upon confirmed feature drift.
• **Centralized Model Registry & Staging Gates:** Transition from file-based model artifacts to an enterprise MLflow Model Registry with staging, production, and archived lifecycle tags, enforced through automated cryptographic model signing and rollback mechanisms.
• **Canary Deployments & Shadow Scoring:** Implement service mesh routing for canary deployments (routing 5% of live traffic to challenger pods) and shadow scoring (duplicating production traffic to compare latency and model agreement without impacting client responses).
• **Dataset Versioning with DVC:** Integrate Data Version Control (DVC) backed by object storage (S3/GCS) to track raw and processed dataset hashes alongside Git commits, reinforcing experimental reproducibility across model development iterations.